Занятие 2. Обучение с учителем: классификация и ансамбли

Логистическая регрессия, метрики, kNN, деревья решений, Random Forest и введение в бустинг

Python scikit-learn классификация метрики ансамбли

Листайте слайды стрелками ← →

Цели занятия

- Понять, как формулируются задачи классификации.
- Разобрать логистическую регрессию как базовый классификатор.
- Научиться интерпретировать метрики: accuracy, precision, recall, F1.
- Понять, почему дисбаланс классов меняет выбор метрики.
- Познакомиться с kNN, деревьями решений и ансамблями.
- Провести практическое сравнение нескольких моделей.

Программа занятия

Теория

- Логистическая регрессия и вероятность класса
- Метрики классификации
- kNN и масштабирование признаков
- Деревья решений
- Бэггинг, Random Forest, бустинг

Практика

- Загрузка данных
- Разделение train/test
- Обучение нескольких моделей
- Сравнение метрик
- Интерпретация результатов

Повторение: обучение с учителем

- Есть объекты с признаками X.
- Есть известные ответы y.
- Модель обучается на парах (X, y).
- Цель — предсказывать y для новых объектов.

$$X \rightarrow \text{модель} \rightarrow \text{предсказание } \hat{y}$$

Регрессия и классификация

Регрессия

- Целевая переменная — число.
- Примеры: цена, спрос, температура.
- Типичные модели: линейная регрессия.

Классификация

- Целевая переменная — категория или класс.
- Примеры: спам/не спам, тип ириса, нормальная/аномальная активность.
- Типичные модели: логистическая регрессия, kNN, деревья.

Пример задачи классификации

Задача: определить тип цветка по измерениям.

Длина лепестка	Ширина лепестка	Длина чашелистика	Ширина чашелистика	Класс
5.1	3.5	1.4	0.2	setosa
6.7	3.0	5.2	2.3	virginica

Здесь признаки — числовые измерения, а целевая переменная — класс.

Логистическая регрессия: идея

- Несмотря на название, это модель классификации.
- Сначала считается линейная оценка:

$$z = w1 * x1 + w2 * x2 + ... + b$$

- Затем эта оценка преобразуется в вероятность класса.

От линейной оценки к вероятности

- Линейный выход z может быть любым числом.
- Вероятность должна быть в диапазоне от 0 до 1.
- Поэтому используют сигмоиду.

$$p = \frac{1}{(1 + e^{-z})}$$

Сигмоида

Что делает сигмоида

- Сжимает числовую оценку в вероятность.
- Отрицательный z → вероятность около 0.
- Положительный z → вероятность около 1.

$$p = \frac{1}{(1 + e^{-z})}$$

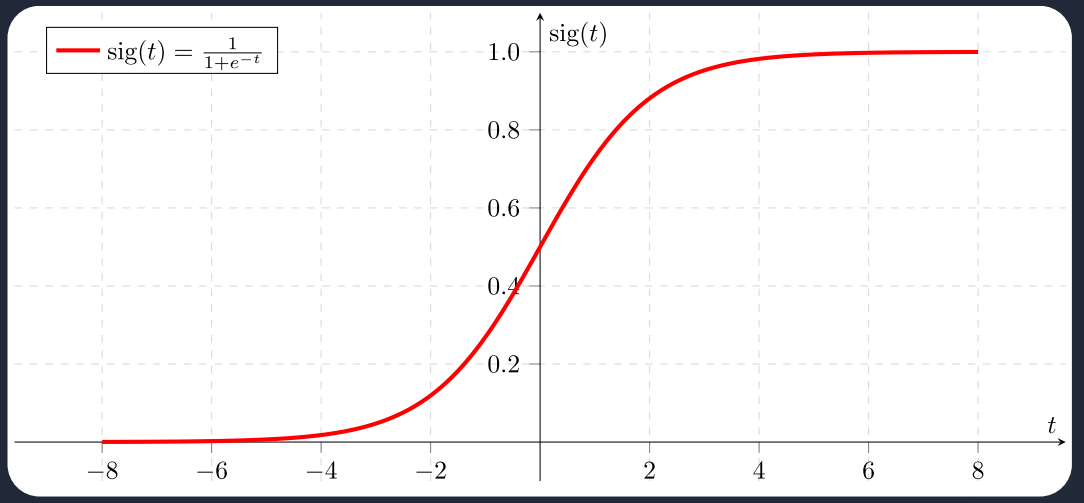


График сигмоиды: по горизонтали z, по вертикали вероятность

Вероятность класса

- Логистическая регрессия может возвращать вероятность:

$$P(y = 1|X)$$

- Например, p = 0.82 означает: модель считает, что объект с вероятностью 82% относится к положительному классу.
- Чтобы получить класс, нужно выбрать порог.

Порог классификации

- По умолчанию часто используют порог 0.5.

$$p >= 0.5 \rightarrow \text{класс } 1$$

$$p < 0.5 \rightarrow \text{класс } 0$$

- Но порог можно менять в зависимости от задачи.

Как выбирают порог

Если дорого пропустить событие

- Можно снизить порог.
- Больше срабатываний.
- Выше recall.

Если дорого ложное срабатывание

- Можно повысить порог.
- Меньше ложных тревог.
- Выше precision.

Логистическая регрессия: плюсы и минусы

Плюсы

- Простая и быстрая модель.
- Возвращает вероятности.
- Хорошая отправная точка.
- Коэффициенты можно интерпретировать.

Минусы

- Линейная граница решения.
- Может плохо ловить сложные зависимости.
- Чувствительна к качеству признаков.

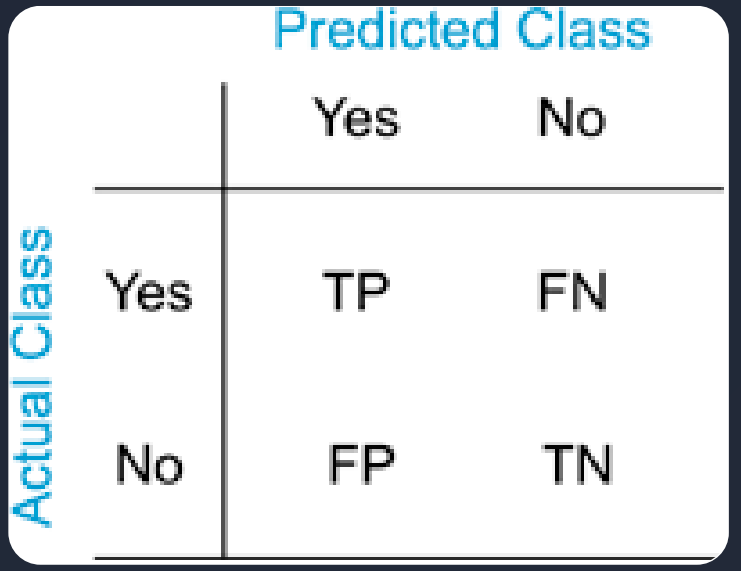
Метрики классификации: зачем они нужны

- Недостаточно просто обучить модель.
- Нужно понять, насколько хорошо она решает задачу.
- Разные ошибки имеют разную цену.
- Метрика должна соответствовать бизнес-логике или требованиям безопасности.

Одна и та же модель может выглядеть хорошей или плохой в зависимости от выбранной метрики.

Матрица ошибок

	Фактический класс 1	**Фактический класс 0**
Предсказан класс 1	True Positive (TP)	False Positive (FP)
Предсказан класс 0	False Negative (FN)	True Negative (TN)



Визуализация матрицы ошибок: можно показать доли или числа

TP, FP, FN, TN

TP

Модель сказала «1», и факт был «1».

FP

Модель сказала «1», но факт был «0». Ложная тревога.

FN

Модель сказала «0», но факт был «1». Пропуск события.

TN

Модель сказала «0», и факт был «0».

Accuracy

$$accuracy = \frac{(TP + TN)}{(TP + FP + FN + TN)}$$

- Доля правильных ответов модели.
- Простая и понятная метрика.
- Хорошо работает, когда классы сбалансированы.
- Может обманывать при сильном дисбалансе классов.

Precision

$$precision = \frac{TP}{(TP + FP)}$$

- Показывает, насколько можно доверять положительным прогнозам.
- Если модель сказала «1», как часто это действительно «1»?
- Важна, когда ложные срабатывания дороги.

Recall

$$recall = \frac{TP}{(TP + FN)}$$

- Показывает, какую долю реальных положительных случаев нашла модель.
- Важна, когда дорого пропустить событие.
- Пример: поиск атак, мошенничества, опасных событий.

F1-score

$$F1 = \frac{2 * precision * recall}{(precision + recall)}$$

- Баланс между precision и recall.
- Полезен, когда важны и ложные срабатывания, и пропуски.
- Часто используется при дисбалансе классов.

Как читать метрики

Метрика	Вопрос
Accuracy	Насколько часто модель права в целом?
Precision	Насколько надежны тревоги модели?
Recall	Насколько полно модель находит важные события?
F1	Каков баланс между надежностью и полнотой?

Пример выбора метрики

Спам-фильтр

Важно не удалить нормальное письмо. Часто смотрят на ложные срабатывания и точность.

Поиск атак

Важно не пропустить атаку. Часто важен высокий recall.

Медицинская диагностика

Цена ошибки высока. Баланс метрик зависит от последствий пропуска и ложной тревоги.

Дисбаланс классов

Проблема

- Одного класса сильно больше, чем другого.
- Пример: 99% нормальных событий и 1% аномалий.
- Модель может всегда отвечать «0» и получать 99% accuracy.
- Но полезность такой модели близка к нулю.



Например, столбики: 0.2% класс 0 и 99% класс 1

Метрики при дисбалансе

- Precision
- Recall
- F1-score
- PR-AUC
- Матрица ошибок

Если целевой класс редкий, лучше оценивать модель именно по этим метрикам.

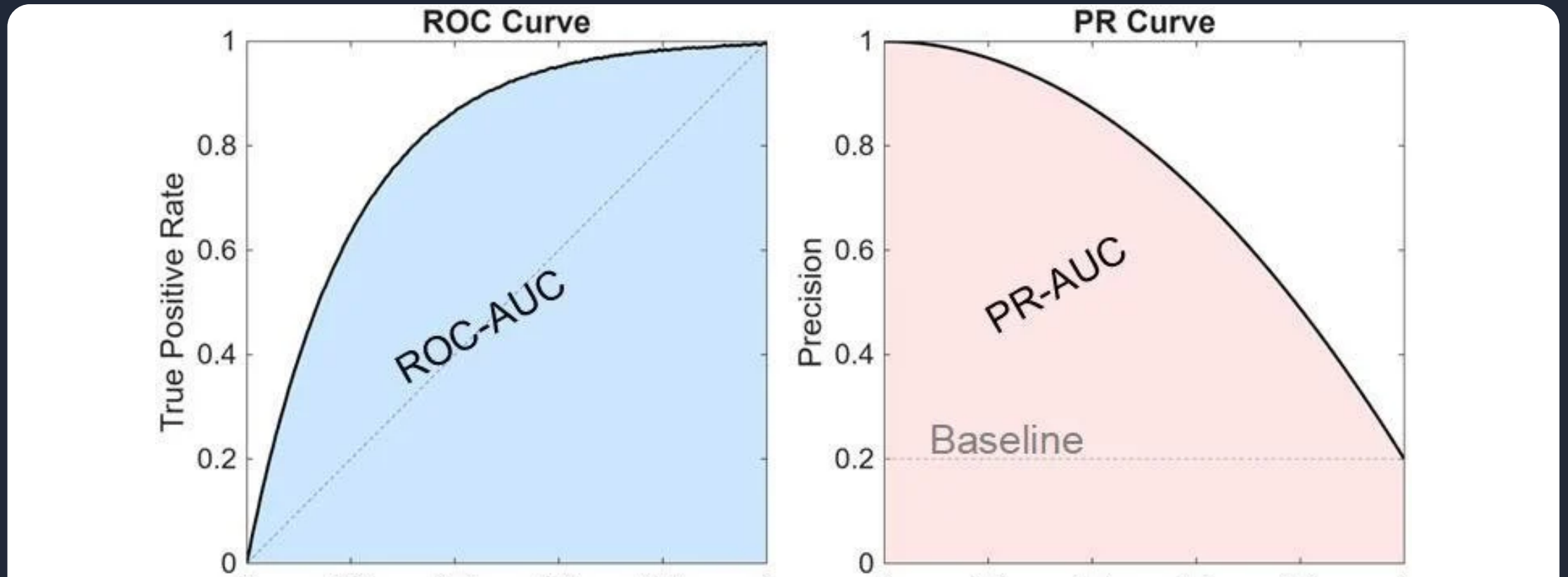
Кратко про ROC-AUC и PR-AUC

ROC-AUC

- Оценивает качество вероятностей модели.
- Показывает, насколько хорошо модель разделяет классы.
- Часто полезна при умеренном дисбалансе.

PR-AUC

- Кривая precision-recall.
- Часто полезнее при сильном дисбалансе.
- Фокусируется на качестве предсказания редкого класса.

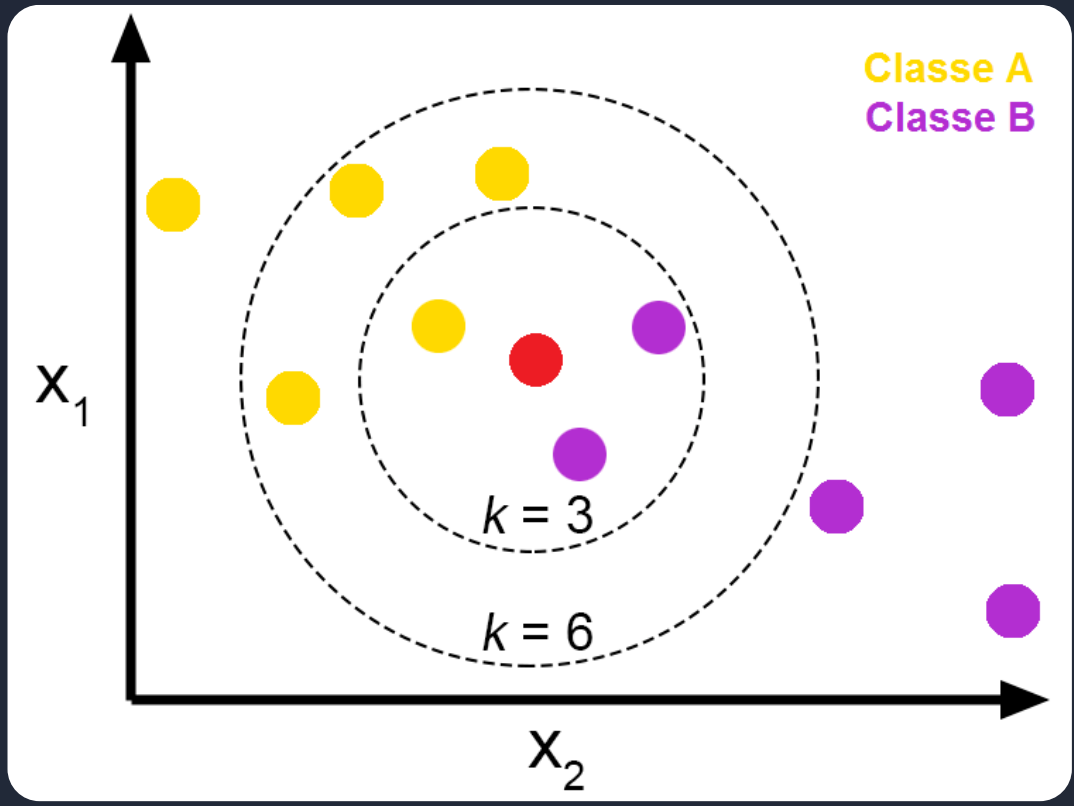


kNN: идея

Основная мысль

- kNN — метод k ближайших соседей.
- Идея: объект похож на свои ближайшие соседи.
- Для классификации класс определяется голосованием соседей.

$$\text{класс} = \text{голосование } k \text{ ближайших объектов}$$

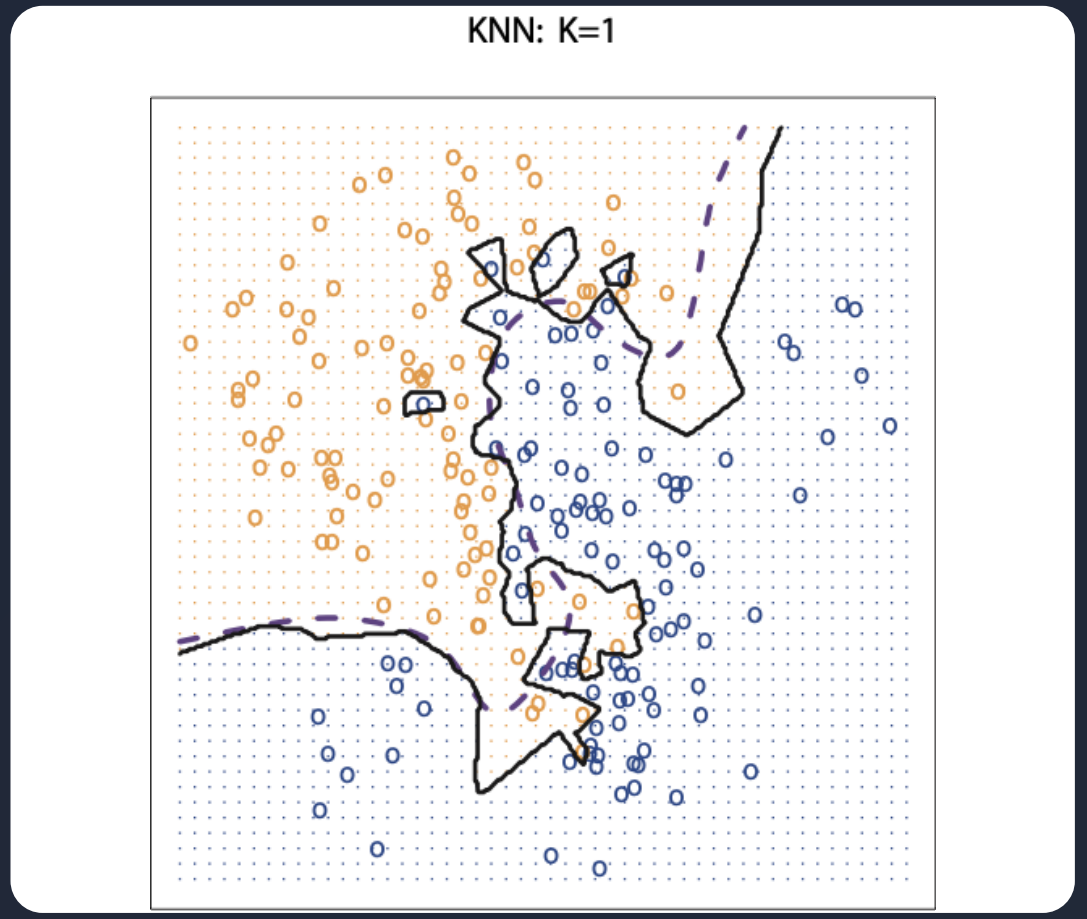


Новая точка и её ближайшие соседи на плоскости признаков

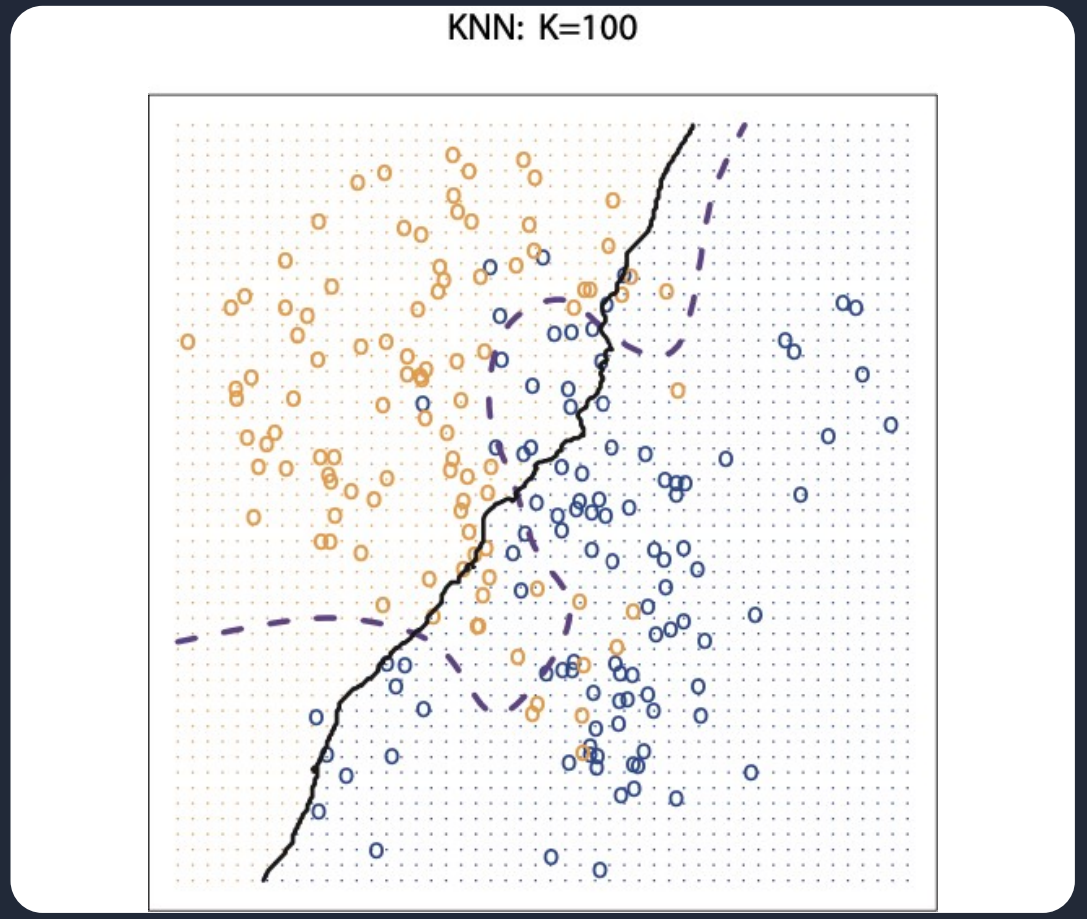
Как работает kNN

1. Выбираем число соседей k.
2. Для нового объекта находим ближайшие обучающие объекты.
3. Смотрим их классы.
4. Присваиваем новый объект классу, который чаще встречается среди соседей.

Выбор k



k = 1: граница изрезанная, риск переобучения



k = 10: граница более сглаженная

Расстояния и масштаб признаков

- kNN использует расстояния между объектами.
- Если один признак измеряется тысячами, а другой — долями единицы, первый будет доминировать.
- Поэтому признаки часто нужно масштабировать.

Для kNN масштабирование признаков особенно важно.

Масштабирование признаков

Зачем

- Привести признаки к сопоставимому масштабу.
- Помочь моделям, основанным на расстояниях.
- Улучшить численную устойчивость и интерпретируемость.

Пример до

зарплата: 120000

возраст: 35

Пример после

зарплата: 0.62

возраст: -0.18

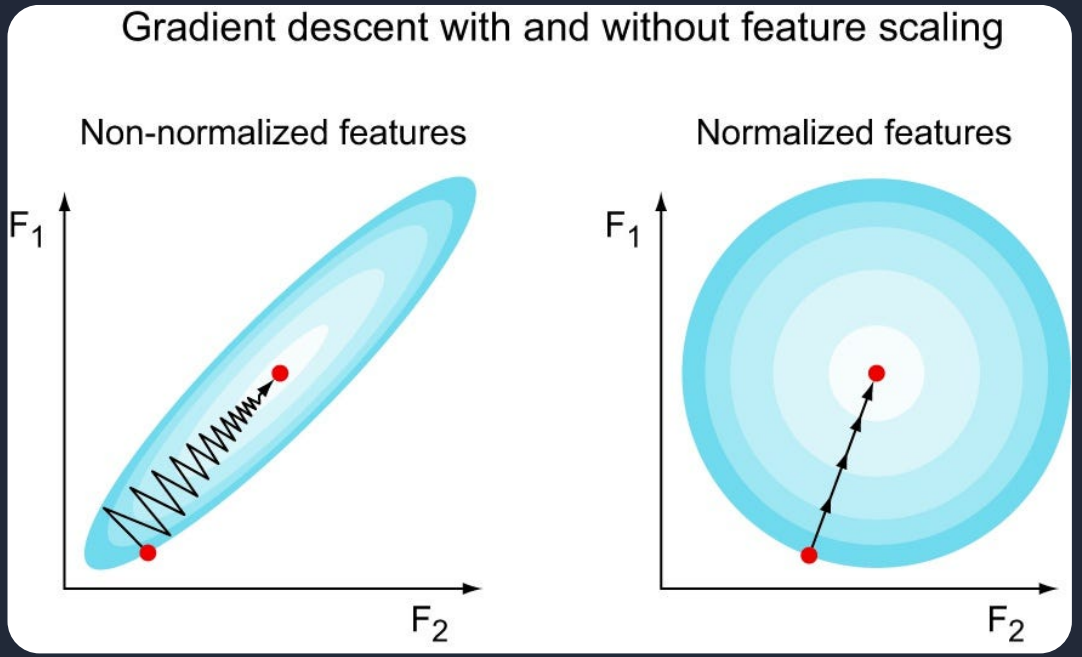


Схема: признаки до масштабирования и после

StandardScaler и MinMaxScaler

StandardScaler

- Приводит признаки к среднему 0 и стандартному отклонению 1.
- Хорош для многих моделей.

MinMaxScaler

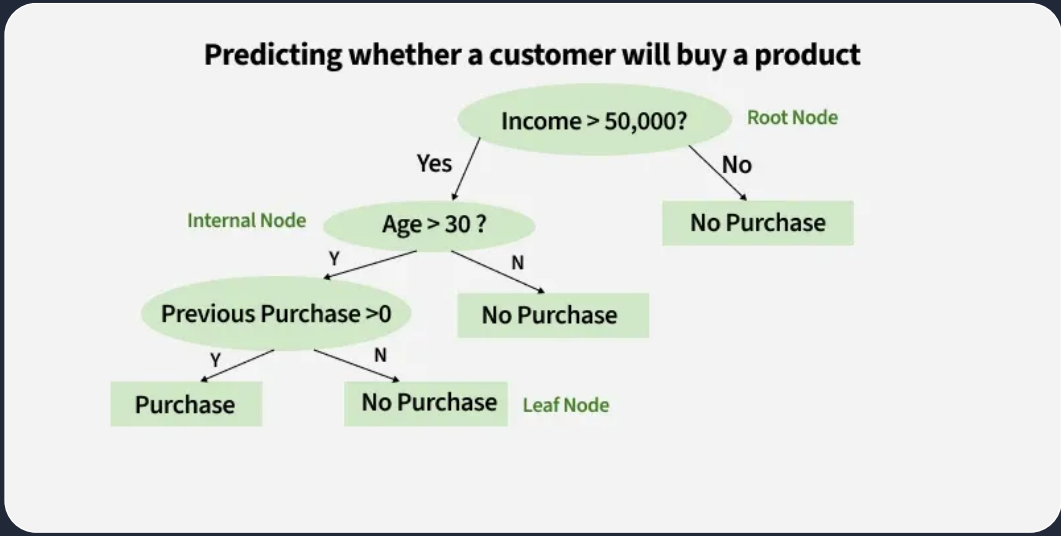
- Масштабирует признаки в диапазон, например от 0 до 1.
- Чувствителен к выбросам.

Деревья решений: идея

Как думать о дереве

- Дерево последовательно задает вопросы о признаках.
- Каждый вопрос делит объекты на группы.
- В листьях дерева находится предсказание класса.

$$\text{если} \text{признак} > \text{порог} \to \text{ветка } 1, \text{иначе} \to \text{ветка } 0$$



Пример дерева решений с вопросами и листьями

Как дерево делает разбиения

1. Дерево выбирает признак.
2. Выбирает порог.
3. Делит данные на две части.
4. Повторяет процесс для получившихся групп.
5. Останавливается по критерию или ограничению.

Критерии качества разбиения

Gini impurity

Оценивает вероятность ошибочной классификации объекта внутри узла.

Entropy

Оценивает неопределенность распределения классов в узле.

Оба критерия помогают дереву выбирать более «чистые» разбиения.

Глубина дерева и переобучение

Слишком глубокое

- Запоминает обучающие данные.
- Хорошо работает на обучении.
- Плохо обобщает на тесте.

Слишком мелкое

- Не улавливает закономерности.
- Может недообучаться.

Глубина — важный гиперпараметр дерева.

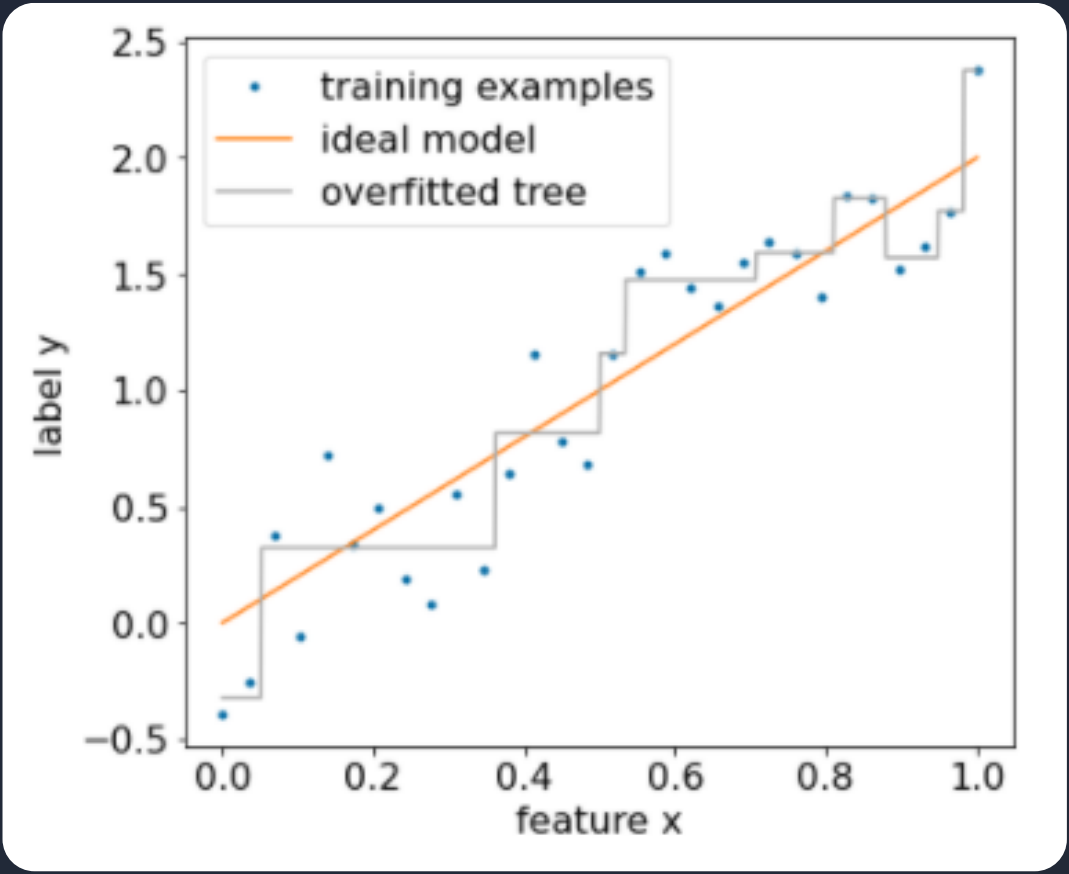


Иллюстрация: недообучение, баланс, переобучение

Важность признаков

- Деревья могут показывать, какие признаки сильнее влияют на разбиения.
- Это помогает интерпретировать модель.
- Но важность признаков нужно интерпретировать осторожно.
- Корреляция признаков может искажать картину.

Деревья: плюсы и минусы

Плюсы

- Понятная логика решений.
- Не требуют обязательного масштабирования.
- Могут работать с нелинейными зависимостями.

Минусы

- Склонны к переобучению.
- Могут быть нестабильными.
- Небольшие изменения данных могут менять дерево.

Ансамбли моделей

- Ансамбль — это несколько моделей, которые работают вместе.
- Идея: несколько слабых или разных моделей могут дать лучший результат, чем одна.
- Ансамбли часто снижают дисперсию и улучшают обобщение.

Одно дерево может ошибаться. Группа деревьев может быть устойчивее.

Бэггинг

Идея

- Bagging = bootstrap aggregating.
- Каждая базовая модель обучается на своей подвыборке данных.
- Затем предсказания усредняются или выбираются голосованием.
- Помогает снизить разброс модели.

много моделей на подвыборках $\rightarrow$ агрегация ответов

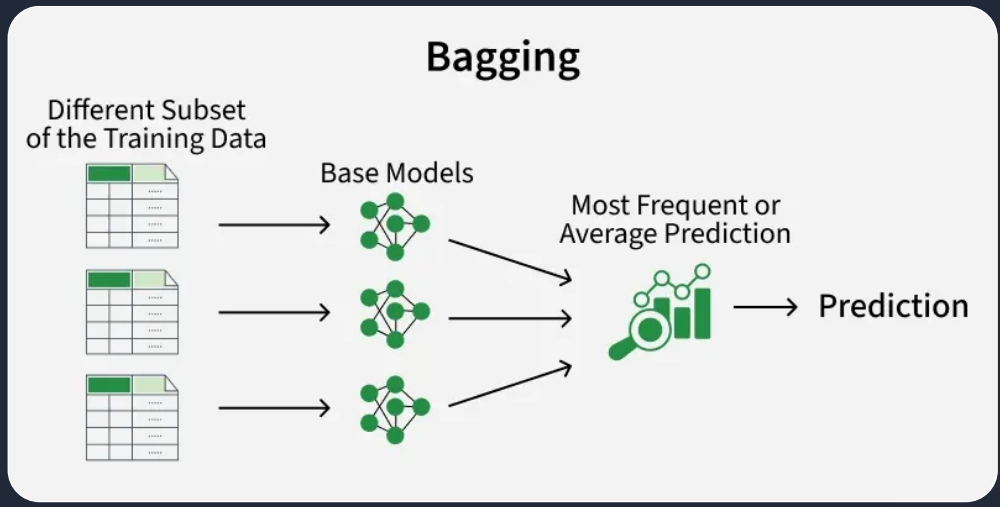


Схема бэггинга: разные подвыборки, разные модели, агрегация

Random Forest

Как устроен

- Random Forest — ансамбль деревьев решений.
- Каждое дерево обучается на случайной подвыборке объектов.
- При разбиениях также используется случайный подмножество признаков.
- Итоговый класс выбирается голосованием деревьев.

Random Forest часто является сильной моделью по умолчанию для табличных данных.

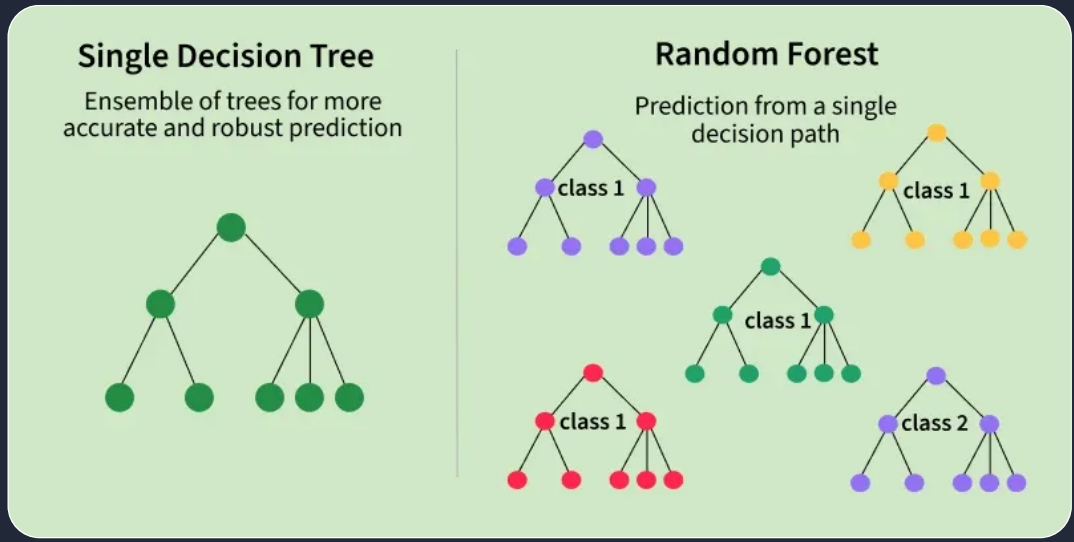


Схема Random Forest: много деревьев и голосование

Гиперпараметры Random Forest

- n_estimators — число деревьев.
- max_depth — максимальная глубина деревьев.
- min_samples_split — минимальное число объектов для разбиения.
- min_samples_leaf — минимальное число объектов в листе.
- max_features — число признаков, рассматриваемых при разбиении.

Бустинг: идея

- Бустинг строит модели последовательно.
- Каждая новая модель старается исправить ошибки предыдущих.
- Итоговый прогноз — взвешенная комбинация моделей.

$$\text{модель } 1 \rightarrow \text{исправляем ошибки} \rightarrow \text{модель } 2 \rightarrow \text{исправляем ошибки} \rightarrow \text{модель } 3\ldots$$

Градиентный бустинг

Основная идея

- Новые модели обучаются на остатках или градиентах ошибки.
- Метод часто дает высокое качество на табличных данных.
- Требует аккуратной настройки гиперпараметров.
- Может переобучаться, если обучать слишком долго.

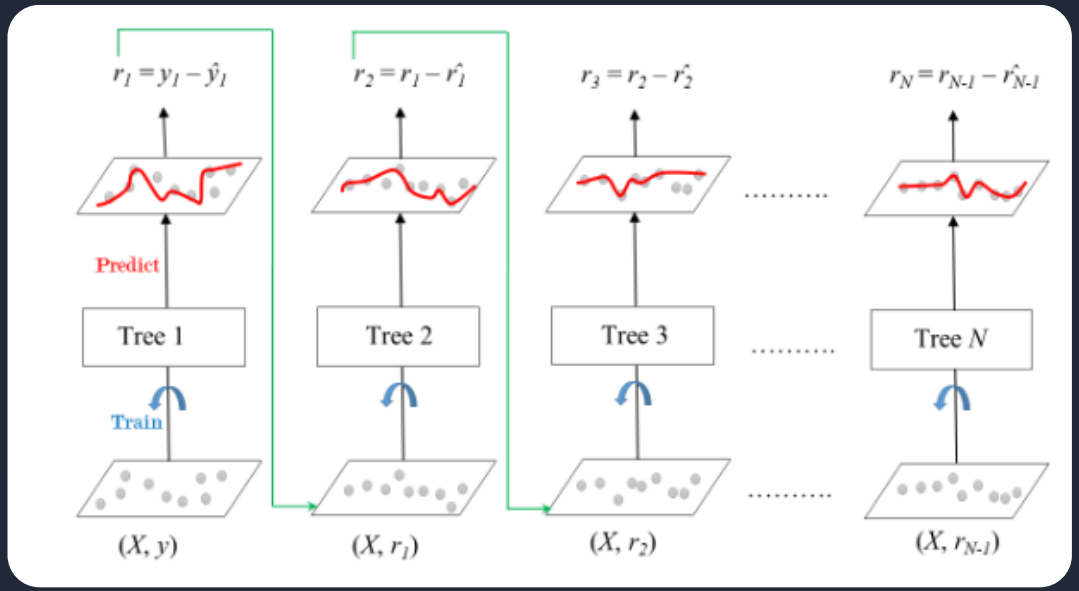


Схема градиентного бустинга: последовательное исправление ошибок

Популярные реализации бустинга

XGBoost

Быстрый и популярный градиентный бустинг.

LightGBM

Часто эффективен на больших табличных данных.

CatBoost

Удобен для данных с категориальными признаками.

В рамках первого знакомства достаточно понимать идею и видеть пример работы.

Практика: постановка эксперимента

- Берем один датасет классификации.
- Разделяем данные на обучение и тест.
- Обучаем несколько моделей.
- Делаем предсказания на тестовой выборке.
- Сравниваем качество по одинаковым метрикам.

Модели для сравнения: логистическая регрессия, kNN, дерево решений, Random Forest.

Рекомендуемые датасеты

Iris

Простой многоклассовый датасет. Хорош для первого знакомства.

Wine

Больше признаков, тоже многоклассовая классификация.

Breast Cancer

Бинарная классификация, удобен для обсуждения метрик.

Типовой pipeline сравнения

1. Загрузить данные.
2. Выделить признаки X и целевую переменную y.
3. Разделить на train и test.
4. При необходимости масштабировать признаки.
5. Обучить модели.
6. Получить предсказания.
7. Посчитать метрики.

Пример кода: сравнение моделей

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, f1_score, classification_report

X, y = load_breast_cancer(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

models = {
    "LogReg": LogisticRegression(max_iter=1000, random_state=42),
    "kNN": KNeighborsClassifier(n_neighbors=5),
    "Tree": DecisionTreeClassifier(max_depth=4, random_state=42),
    "RandomForest": RandomForestClassifier(n_estimators=200, random_state=42),
}

for name, model in models.items():
    if name == "kNN" or name == "LogReg":
        model.fit(X_train_scaled, y_train)
        preds = model.predict(X_test_scaled)
    else:
        model.fit(X_train, y_train)
        preds = model.predict(X_test)

    print(name)
    print("accuracy:", accuracy_score(y_test, preds))
```

Интерпретация результатов

- Сравниваем модели только на тестовой выборке.
- Смотрим не только на accuracy, но и на precision, recall, F1.
- Проверяем, не переобучилось ли дерево.
- Смотрим, помогает ли масштабирование для kNN и логистической регрессии.
- Оцениваем, соответствует ли метрика задаче.

Хороший результат — не только высокая метрика, но и понимание, почему модель так работает.

Итоги и связь с MLSecOps

- Логистическая регрессия — сильная базовая модель для классификации.
- Метрика должна соответствовать цене ошибок.
- kNN требует масштабирования признаков.
- Деревья интерпретируемы, но склонны к переобучению.
- Random Forest часто улучшает устойчивость и качество.
- Бустинг может дать высокий результат, но требует аккуратности.

Для MLSecOps важно: контролировать качество данных, воспроизводимость обучения, корректность валидации и поведение модели в эксплуатации.